NLP and Large Language Models

VL06: Systems & Efficiency

Course 8,862 | 6 ECTS | FS2026

Tuesday 14:15–16:00

Prof. Dr. Siegfried Handschuh

DS-NLP · Universität St. Gallen

Where We Are: The Course Navigator

Week	Topic	Notes
1	Introduction	✓
2	Tokenization & Data	✓
3	Transformer Architecture & Attention	✓
4	Training Fundamentals	✓
5	Scaling Laws	✓
6	Systems & Efficiency	*YOU ARE HERE*

Today's Milestone:

You know **what** to build and **how big**. Today you learn how to make the GPU **actually go fast**.

Semester Break & Next Lecture

- **Semester Break:** 30.03. – 12.04.2026
- **Next lecture:** 14 April, same time, Square

14 April

- *Lecture covered by Götz-Henrik* (Teaching Assistant)
- Topic: **Inference & Decoding**
- **Questions:** please contact Götz-Henrik

Agenda: Making the GPU Go Fast

1. GPU Anatomy 

- CPU vs GPU
- Memory Hierarchy
- The Memory Wall

2. Making It Fast 

- Compute vs Memory Bound
- Efficient Attention (Tiling)
- Mixed Precision on V100

3. Scaling Out 

- DDP: Your Main Tool
- Profiling & Optimization
- Pre-Training Checklist

Learning Objectives

1. GPU Architecture

Explain **why** GPUs are fast for deep learning and identify the **memory hierarchy** bottleneck.

2. Compute vs. Memory Bound

Distinguish compute-bound from memory-bound operations and explain why **tiled attention** avoids the T^2 bottleneck.

3. Mixed Precision on V100

Apply **FP16 + GradScaler** correctly and calculate the **N×16 bytes** memory budget.

4. Distributed Training

Set up **DDP** on the DGX-2 and use **profiling** to identify bottlenecks before the full training run.

Agenda

1. GPU Anatomy
2. Making It Fast
3. Scaling Out
4. Summary & Outlook

GPU Anatomy

Why GPUs Make LLMs Possible


NVIDIA

CPU

Control Unit · Branch Prediction · Out-of-order

Core 1
ALU · FPU · L1/L2

Core 2
ALU · FPU · L1/L2

Core 3
ALU · FPU · L1/L2

Core 4
ALU · FPU · L1/L2

Core 5
ALU · FPU · L1/L2

Core 6
ALU · FPU · L1/L2

Core 7
ALU · FPU · L1/L2

Core 8
ALU · FPU · L1/L2

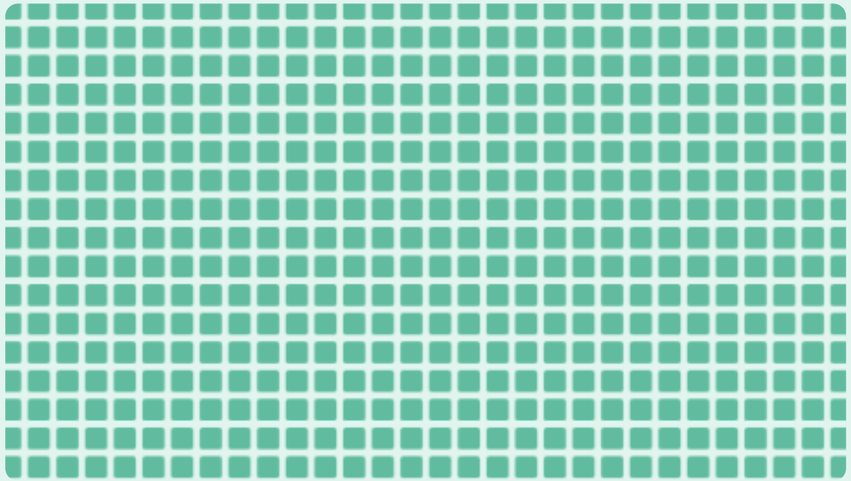
L3 Cache · shared · ~16 MB

8 cores · latency-optimized · ~4 GHz

vs

GPU — NVIDIA V100

5.120 CUDA Cores
Massively parallel processing



640 Tensor Cores
Mixed-Precision · AI acceleration

HBM2 Memory · 32 GB · 900 GB/s

5,120 CUDA · 640 Tensor · 900 GB/s · throughput-optimized

CPU vs. GPU: Two Different Philosophies [1]

CPU: Latency Optimized

- **Few powerful cores** (8–64)
- Large caches, branch prediction
- Each thread finishes **fast**

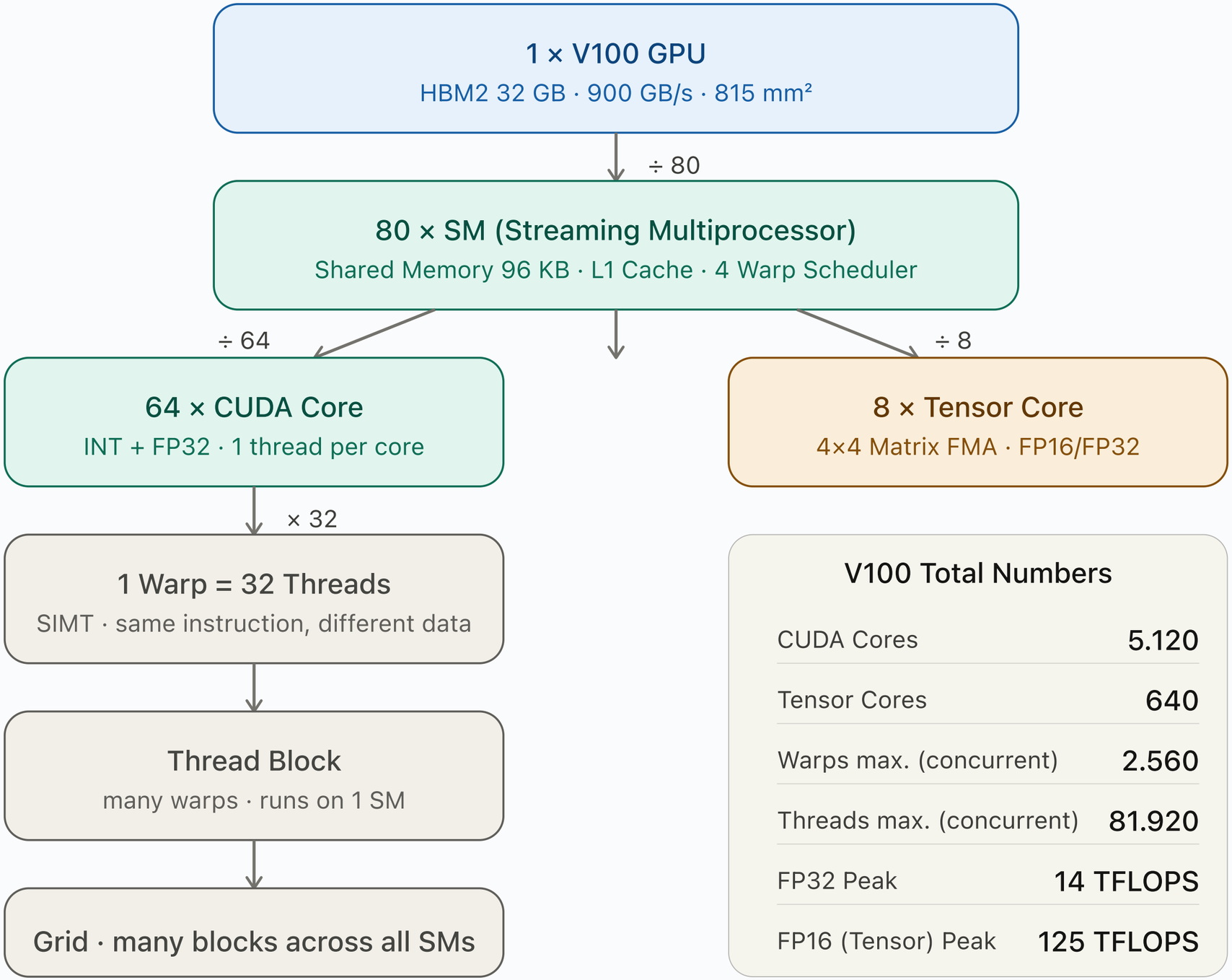
Good for: Complex sequential logic, OS tasks, branching code.

GPU: Throughput Optimized

- **Thousands of simple cores**
- Tiny caches, no branch prediction
- Many threads run **simultaneously**

Good for: Matrix multiply, convolutions, the same operation on many data points.

> ***Deep learning is 90% matrix multiplication.*** *GPUs are designed to do exactly that.* [1]

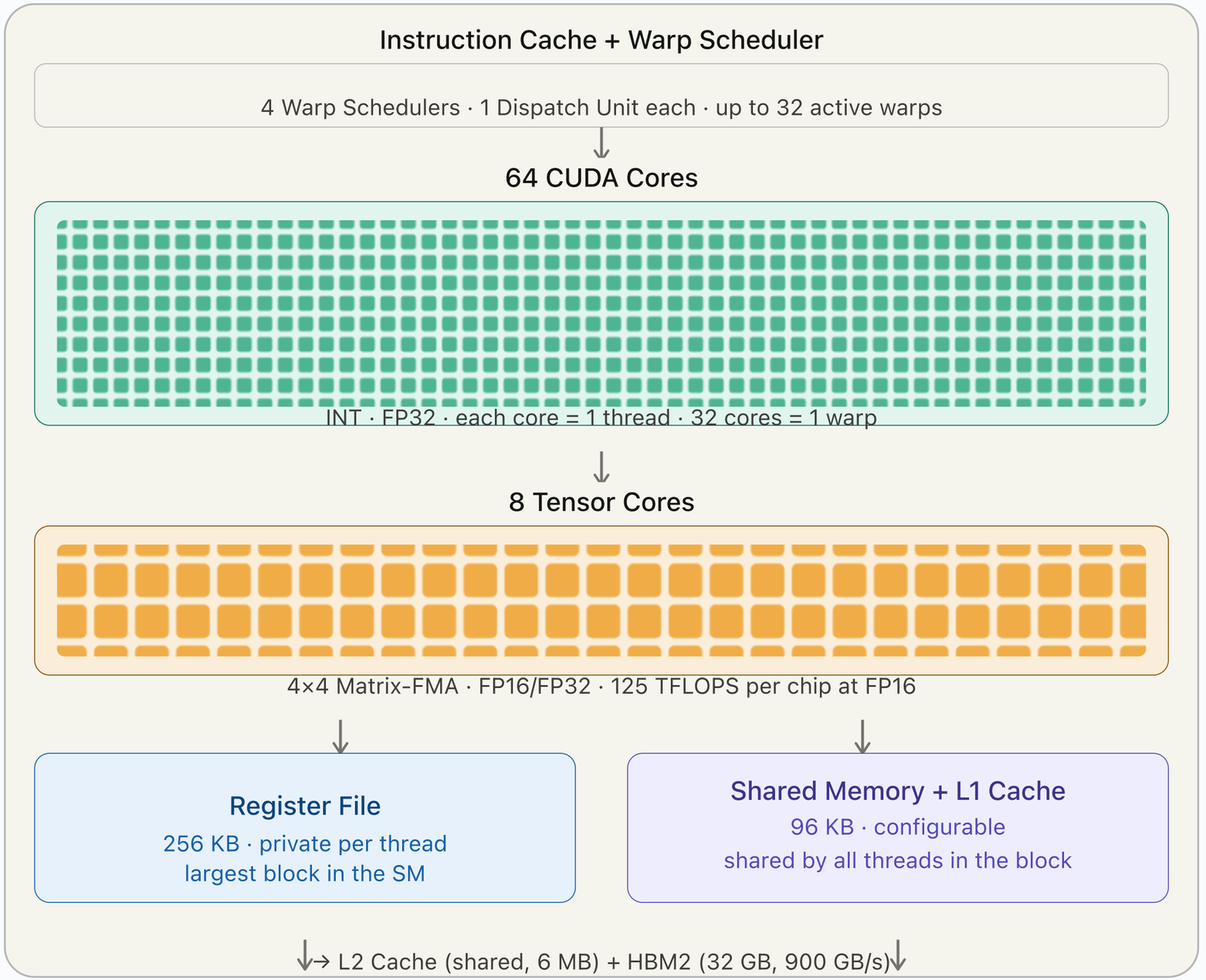


V100 Total Numbers

CUDA Cores	5.120
Tensor Cores	640
Warps max. (concurrent)	2.560
Threads max. (concurrent)	81.920
FP32 Peak	14 TFLOPS
FP16 (Tensor) Peak	125 TFLOPS

V100 Streaming Multiprocessor (SM)

1 of 80 SMs on the chip



Anatomy of a GPU: Streaming Multiprocessors [1,2]

The Building Blocks

A GPU is a collection of **SMs** (Streaming Multiprocessors):

Component	V100	A100	H100
SMs	80	108	132
FP32 cores / SM	64	64	128
Tensor Cores / SM	8	4	4
Shared Memory / SM	96 KB	164 KB	228 KB

Each SM runs **blocks** of threads independently.

Tensor Cores: The Secret Weapon

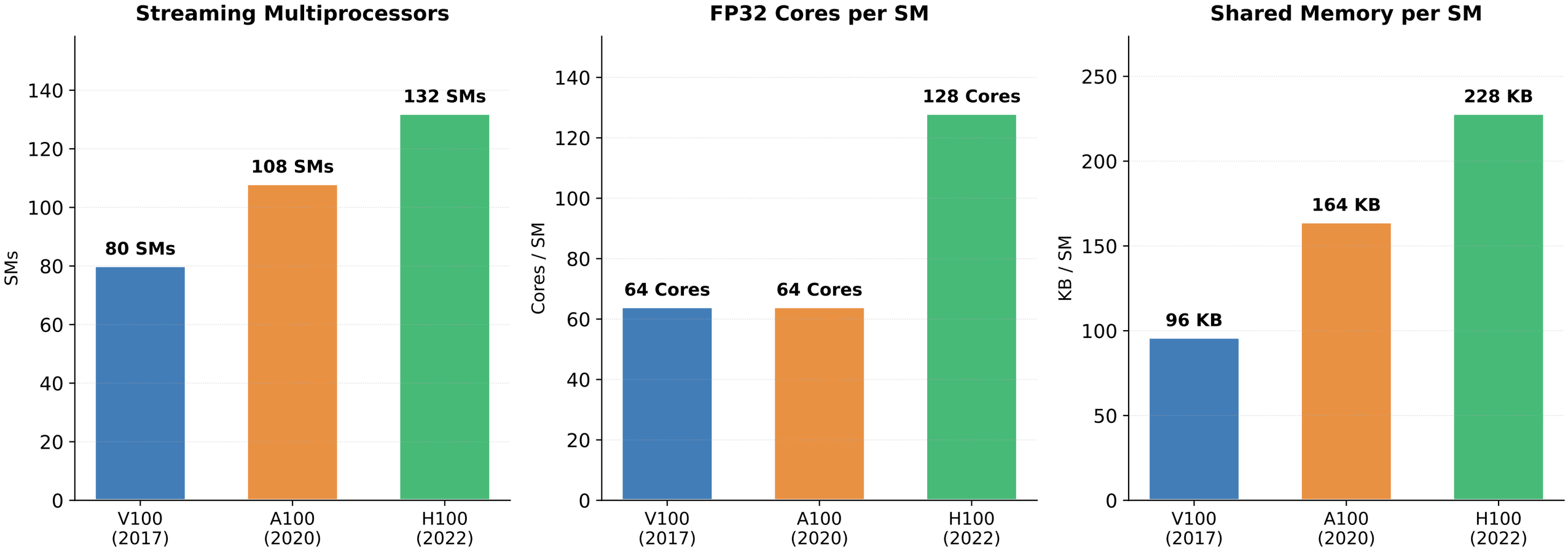
Since the V100 (Volta, 2017), GPUs have **Tensor Cores**, i.e. specialized circuits for matrix multiplication.

Matmul on Tensor Cores is **>10× faster** than on regular CUDA cores. [1]

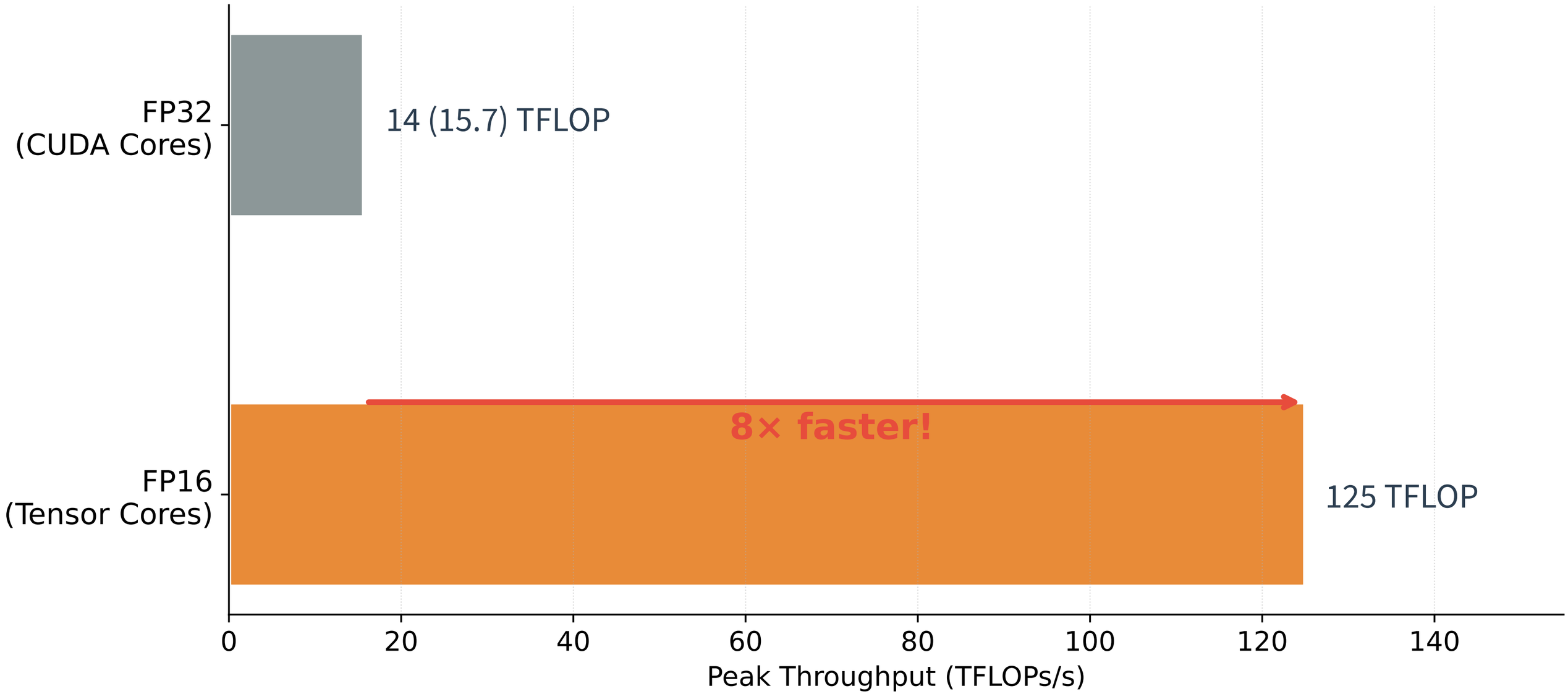
Operation	V100 TFLOPs
FP32 (CUDA cores)	14 (15.7)
FP16 (Tensor Cores)	**125**

Use Tensor Cores → use mixed precision!

GPU Generation Comparison: Streaming Multiprocessors

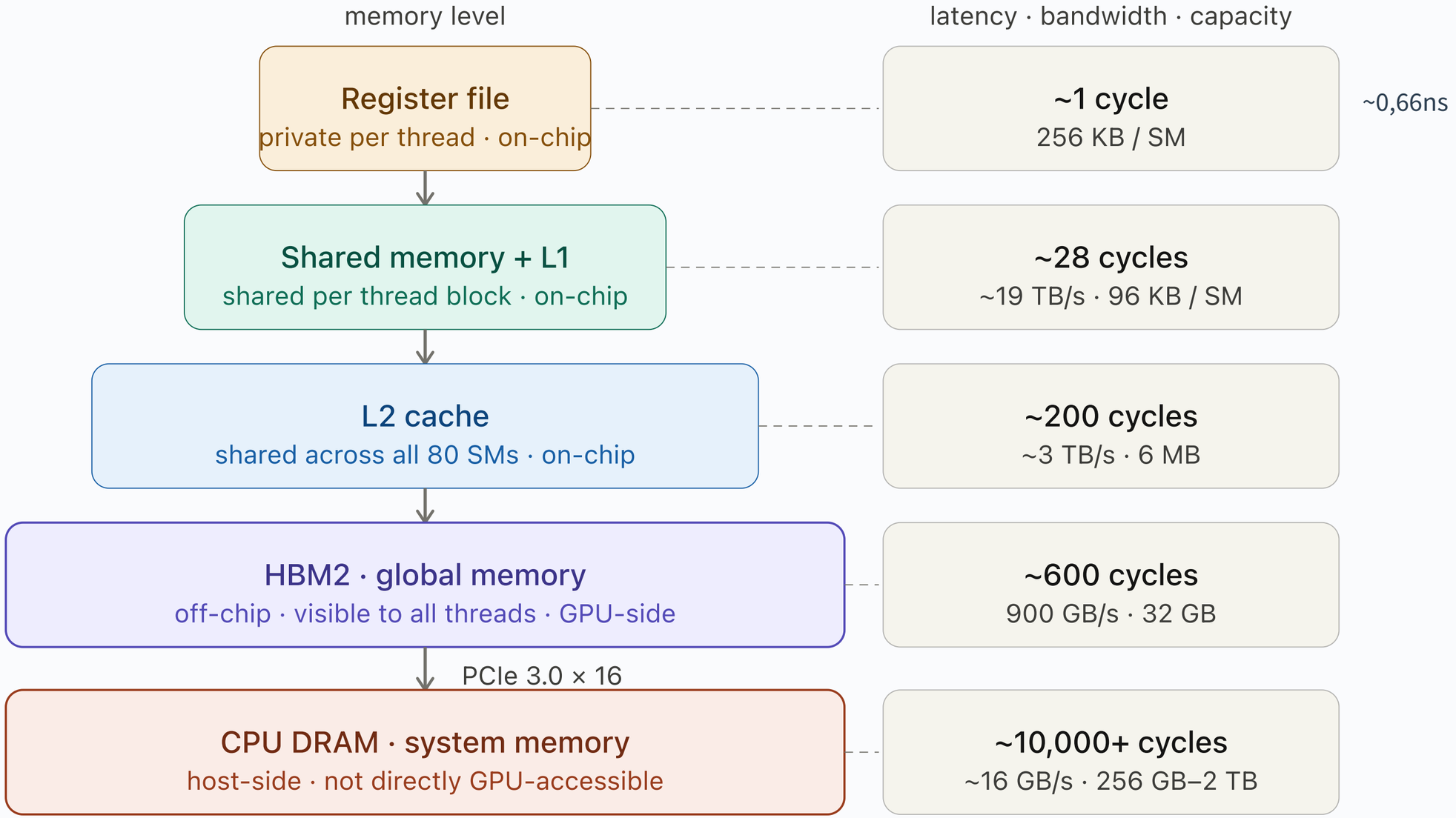


V100: Tensor Cores Unlock 8× Matmul Speed



Use mixed precision → use Tensor Cores!

V100 full memory hierarchy



bar width = relative capacity

PCIe bandwidth ~56× slower than HBM2
→ minimize host↔device transfers

The Memory Hierarchy: Speed vs. Size [1,2]

Closer = Faster, Smaller

Memory	Size (V100)	Latency	Speed
Registers	~256 KB/SM	1 cycle	Fastest
Shared Mem / L1	96 KB/SM	~23 cycles	Fast
L2 Cache	6 MB	~200 cycles	Medium
HBM (VRAM)	**32 GB**	**~290 cycles (~210 ns)**	Slow
CPU RAM	256+ GB	~1000+ cycles (~720 ns)	Slower

SRAM is ~100× more expensive but **~8× faster** than HBM.

The Training Memory Map

Where does your PikoGPT (~40M) live?

What	Where	Size
Weights (FP32)	HBM	153 MB
Gradients (FP32)	HBM	153 MB
Adam states (FP32)	HBM	305 MB
Activations	HBM	**variable**
Tile during matmul	SRAM	~96 KB

Everything lives in **slow HBM**. The art is getting it into **fast SRAM** efficiently.

The Memory Wall [1]

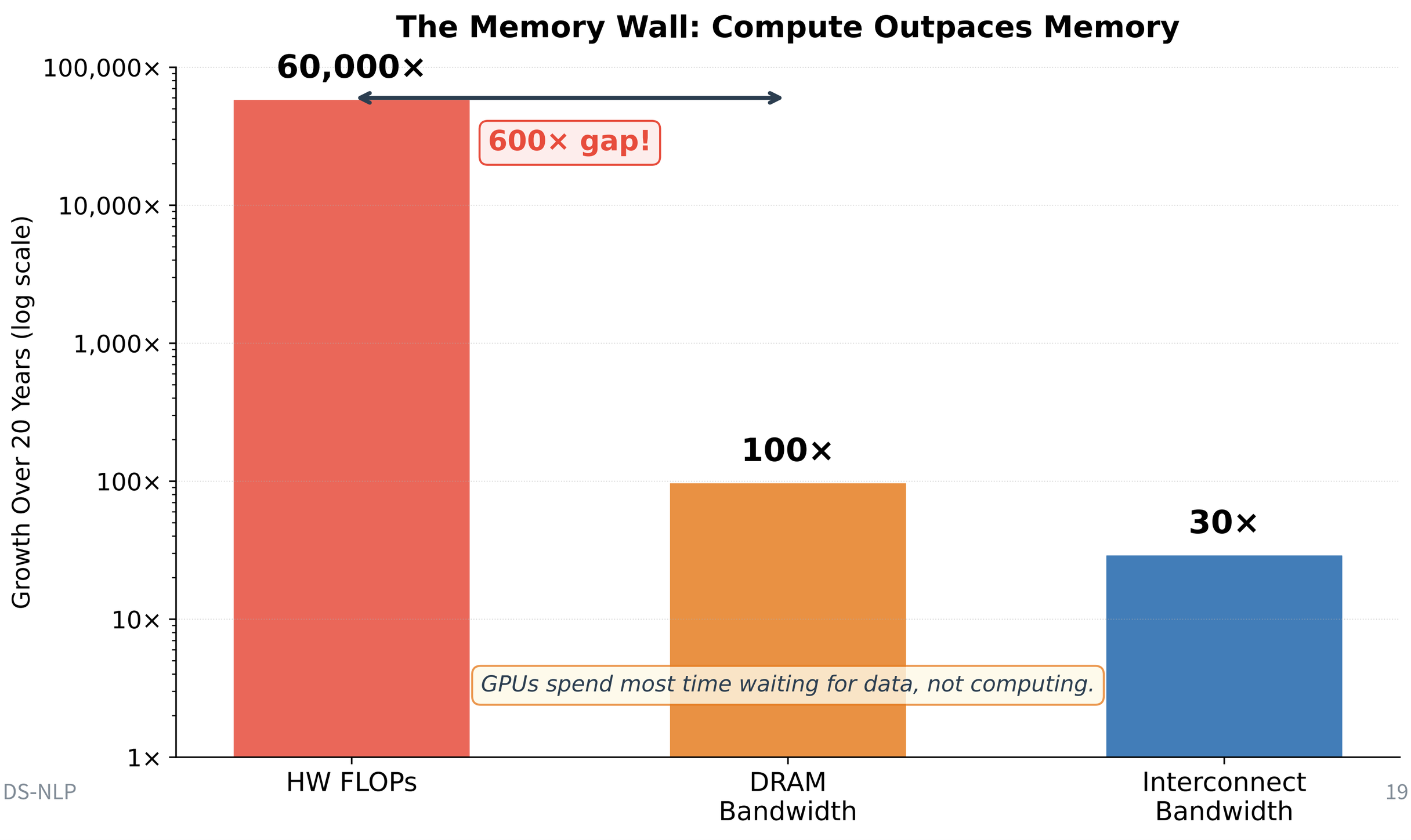
In the last 20-years GPU compute grew

- **60,000×**, but
- memory bandwidth only **100×**. [1]

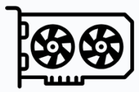
Tensor Cores are literally starving for data.

The Paradigm Shift:

The single most impactful optimization today is *reducing memory traffic**, not accelerating math.


The Memory Wall: Compute Outpaces Memory
Growth Over 20 Years (log scale)
100,000×
10,000×
1,000×
100×
10×
1×
60,000×
600× gap!
100×
30×
GPUs spend most time waiting for data, not computing.
HW FLOPs
DRAM Bandwidth
Interconnect Bandwidth

Take-Home Message: GPU Anatomy

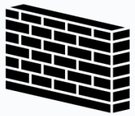


GPUs = Massively Parallel Matmul Machines

Thousands of simple cores (SIMT), optimized for throughput. Tensor Cores make FP16 matmul **8× faster** than FP32.

Memory hierarchy is everything

Registers → SRAM (fast, tiny) → HBM (slow, big). Training data lives in HBM. The art is minimizing round-trips.



The Memory Wall

Compute grows 600× faster than bandwidth. GPUs spend most of their time *waiting for data*, not computing.

Every optimization = less HBM traffic

Tiled attention, kernel fusion, torch.compile, they all reduce memory movement. That's the common thread.

Quick Check: GPU Anatomy

1. The V100 has ___ SMs, each with ___ Tensor Cores.

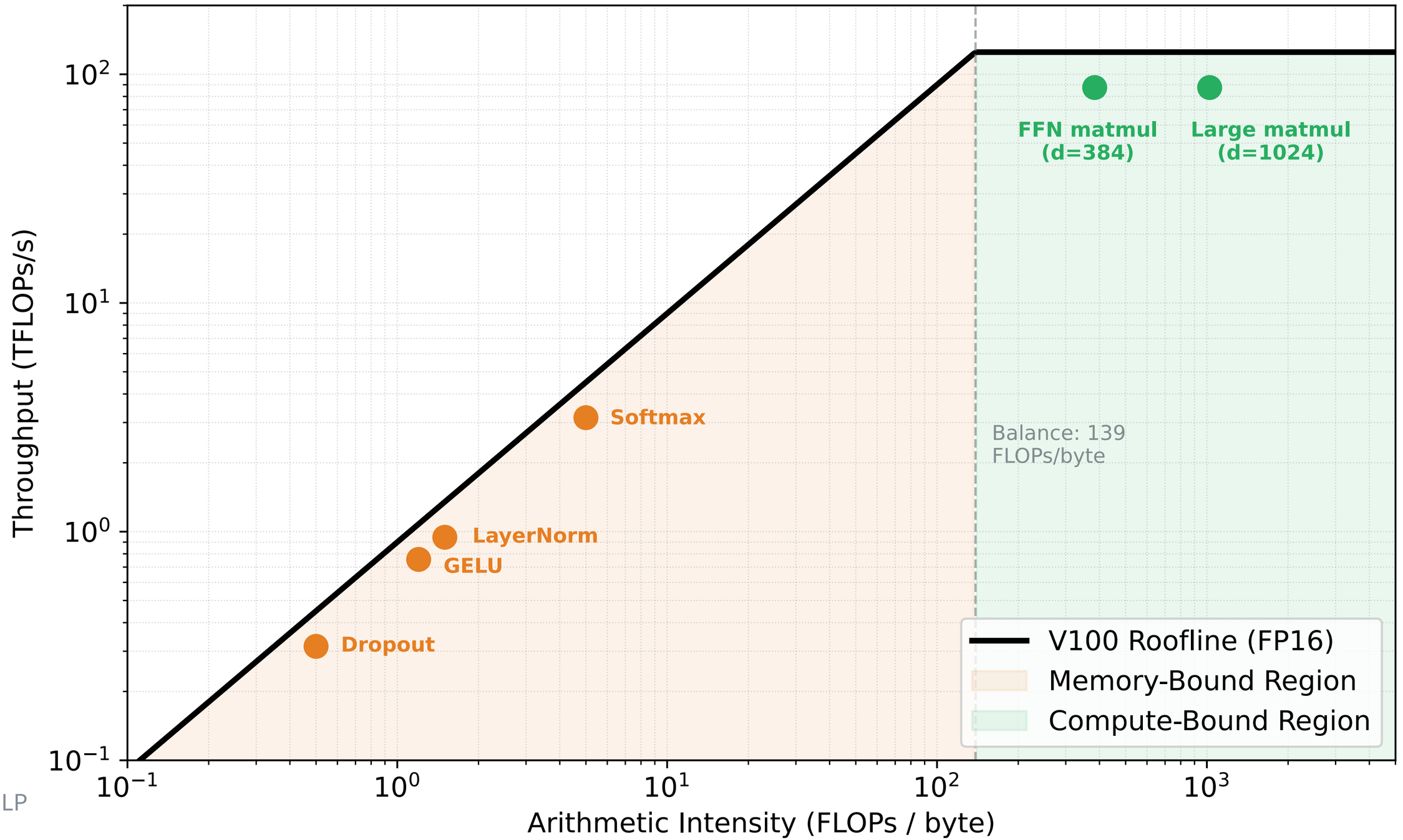
2. FP16 Tensor Cores are ___× faster than FP32 CUDA cores.

3. Which memory is fastest: HBM, SRAM, or Registers?

4. The memory wall means: compute grew ___×, bandwidth only ___×.

Making It Fast

From Memory-Bound to Compute-Bound


V100 Roofline Model: Where Is Your Bottleneck?
Throughput (TFLOPs/s)
Arithmetic Intensity (FLOPs / byte)
10^{2}
10^{1}
10^{0}
10^{-1}
10^{-1}
10^{0}
10^{1}
10^{2}
10^{3}
FFN matmul (d=384)
Large matmul (d=1024)
Softmax
LayerNorm
GELU
Dropout
Balance: 139 FLOPs/byte
V100 Roofline (FP16)
Memory-Bound Region
Compute-Bound Region

Compute-Bound vs. Memory-Bound [3]

The Key Question

For every operation, ask:

Is the GPU waiting for data, or is it computing?

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes loaded from HBM}}$$

Operation	Arith. Intensity	Bottleneck
Large matmul	**High** (~1000)	Compute
LayerNorm	Low (~1)	Memory
Softmax	Low (~5)	Memory
ReLU / GELU	Low (~1)	Memory
Dropout	Low (~0.5)	Memory

The V100 Balance Point

On a V100: 125 TFLOPs FP16, 900 GB/s HBM bandwidth.

$$\text{Balance point} = \frac{125 \times 10^{12}}{900 \times 10^{9}} \approx 139 \text{ FLOPs/byte}$$

If your operation does **<139 FLOPs per byte loaded**, it's **memory-bound**.

Most non-matmul operations in a Transformer are memory-bound!

Implication: Making element-wise ops faster doesn't help. Instead: **fuse operations** to reduce HBM round-trips.

Efficient Attention: The "Online Softmax" Breakthrough [4]

🍾 The Old Way: Standard Attention

The Brute Force Grid

Standard attention calculates the *entire* relationship grid all at once.

- It builds a massive $T \times T$ matrix and saves it to the GPU's slow, main memory (HBM).
- **The Problem:** Memory size explodes exponentially, $O(T^2)$

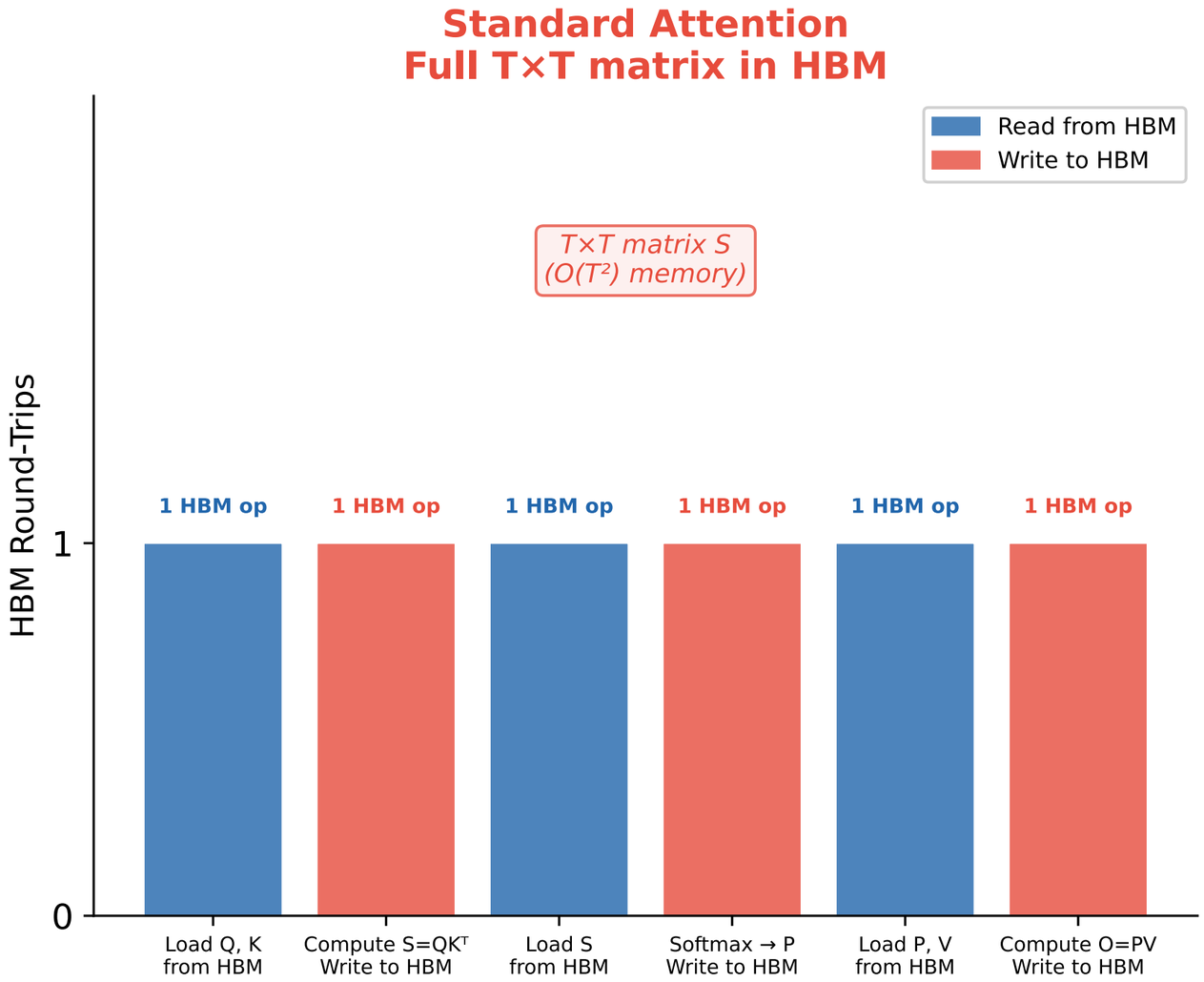
🚀 The Fix: Memory-Efficient (Tiled)

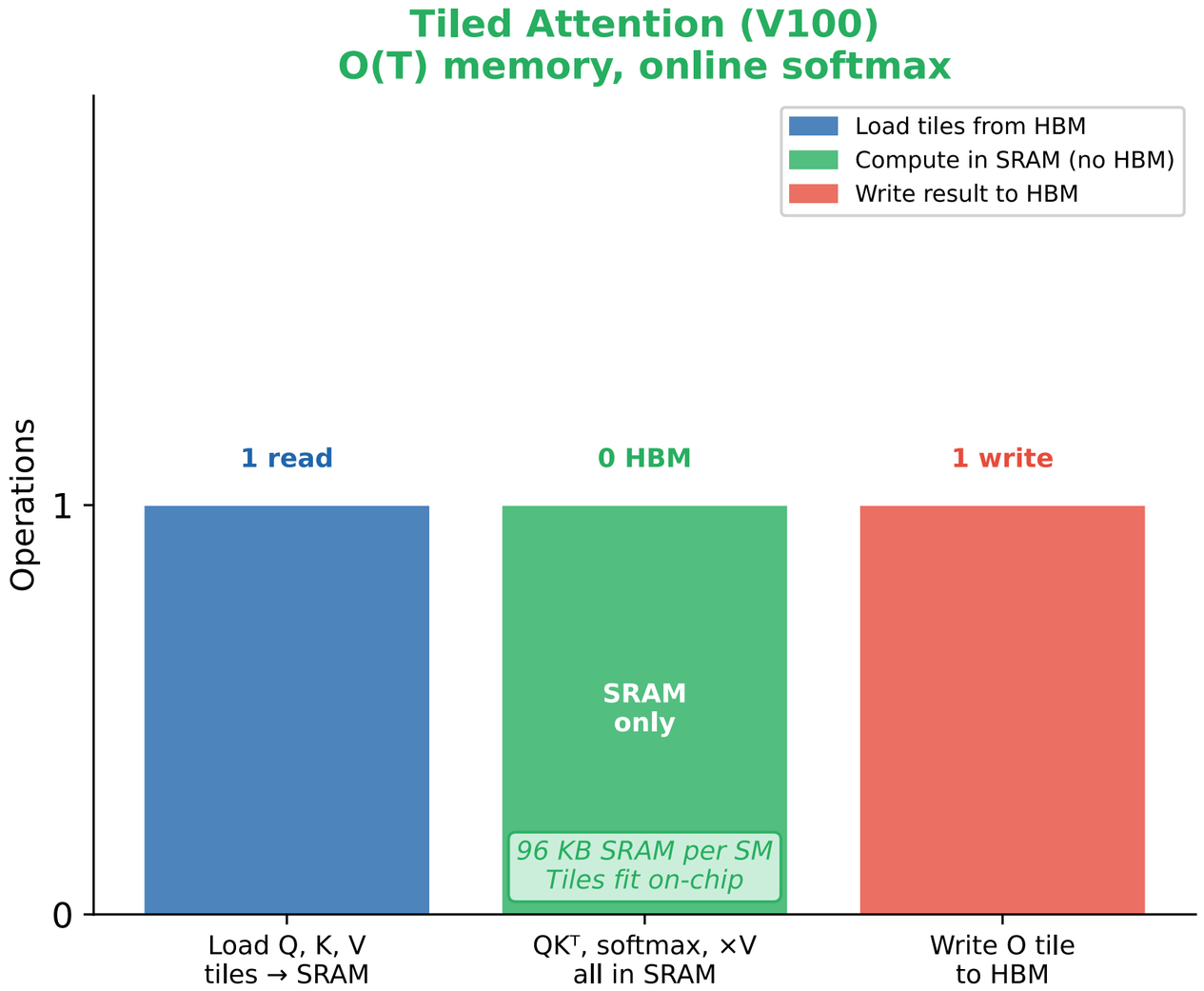
The «Running Tally»

Instead of a giant grid, the GPU loads small «tiles» of text into its ultra-fast, tiny workspace (SRAM).

- **The Magic (Online Softmax):** It calculates relationships *on the fly* by keeping a running tally. It tracks the "max score" so the giant grid never exists.
- **The Result:** Memory usage drops to just $O(T)$

Standard vs Tiled Attention: HBM Traffic on V100





V100 (CC 7.0): PyTorch scaled_dot_product_attention → memory-efficient backend • FlashAttention v2 requires CC ≥ 7.5 (Turing+)

Kernel Fusion: Reducing HBM Round-Trips [3]

The Problem: Pointwise Operations

A Transformer layer has many small operations after the matmul:

Naive execution (5 HBM round-trips):

Step	Operation	HBM Access
1	Matmul QK^T	Read QK, Write S
2	Mask	Read S, Write S'
3	Softmax	Read S', Write P
4	Dropout	Read P, Write P'
5	Matmul PV	Read P', Write O

Each step loads from HBM, does trivial math, writes back.

The Fix: Fuse Into One Kernel

Fused execution (1 HBM round-trip):

Load Q, K, V once → do everything in SRAM → write O once.

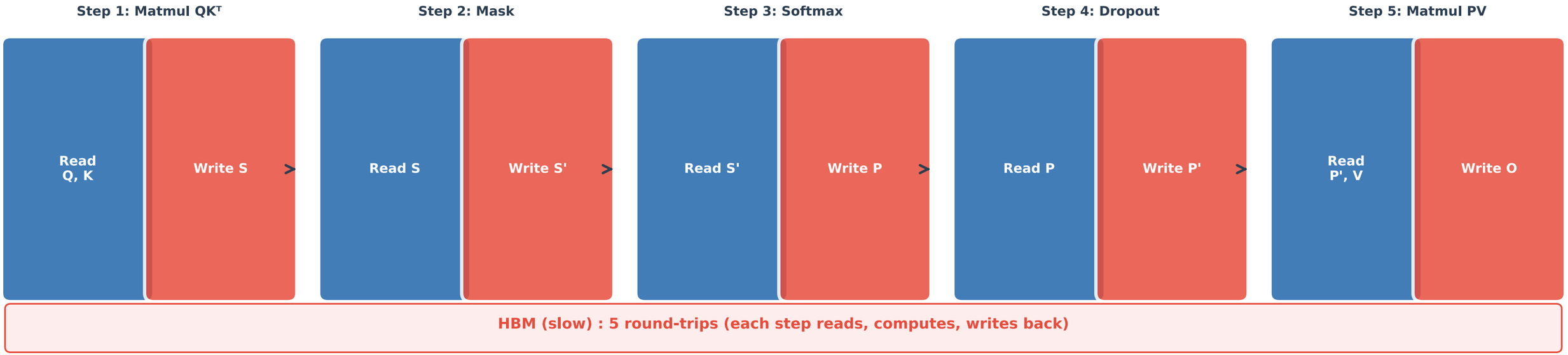
Approach	HBM R/W	How
Naive PyTorch	5×	One kernel per op
`torch.compile`	~2-3×	Auto-fusion
Efficient Attention	**1×**	Tiled CUDA kernel

`torch.compile` is the easiest win.
Efficient attention is the biggest.

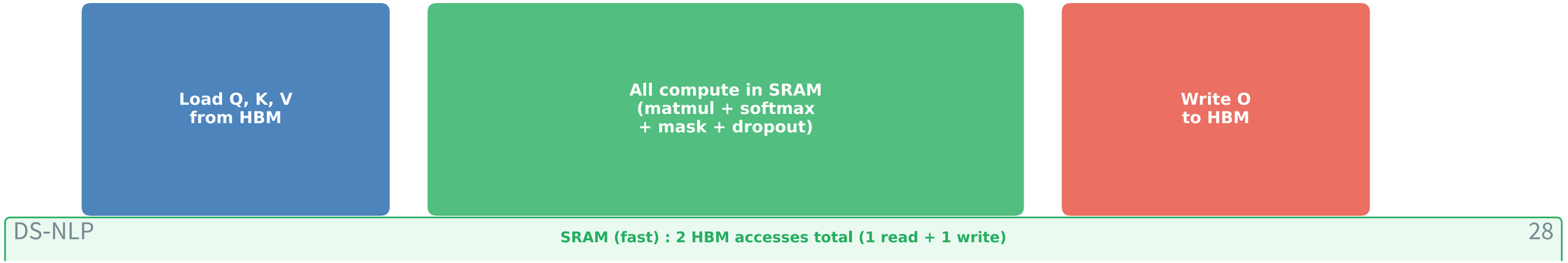
In the exercise: You will add `torch.compile` to your training script. It's a single function call that triggers automatic kernel fusion.

Kernel Fusion: Reducing HBM Round-Trips

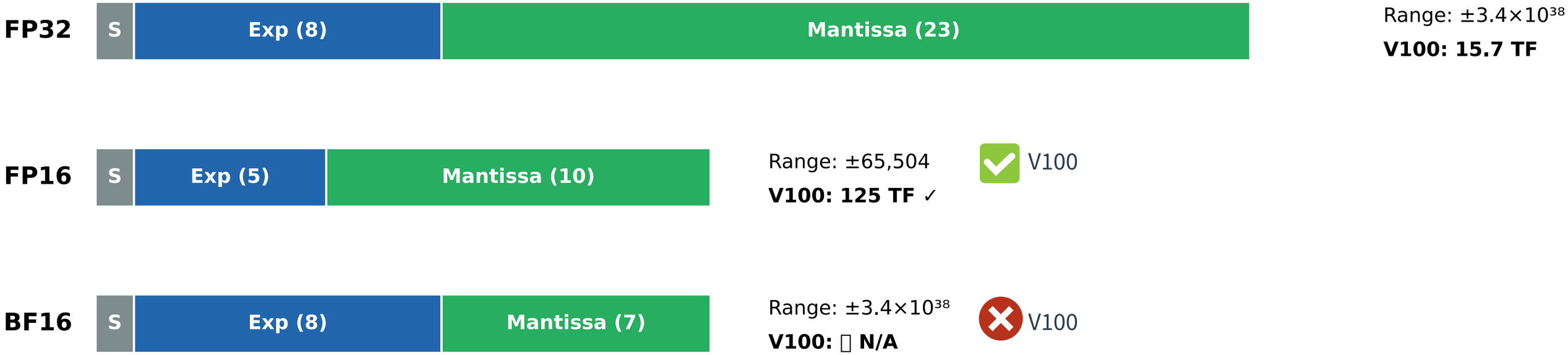
Naive Execution: 5 Steps, Each a Separate HBM Round-Trip



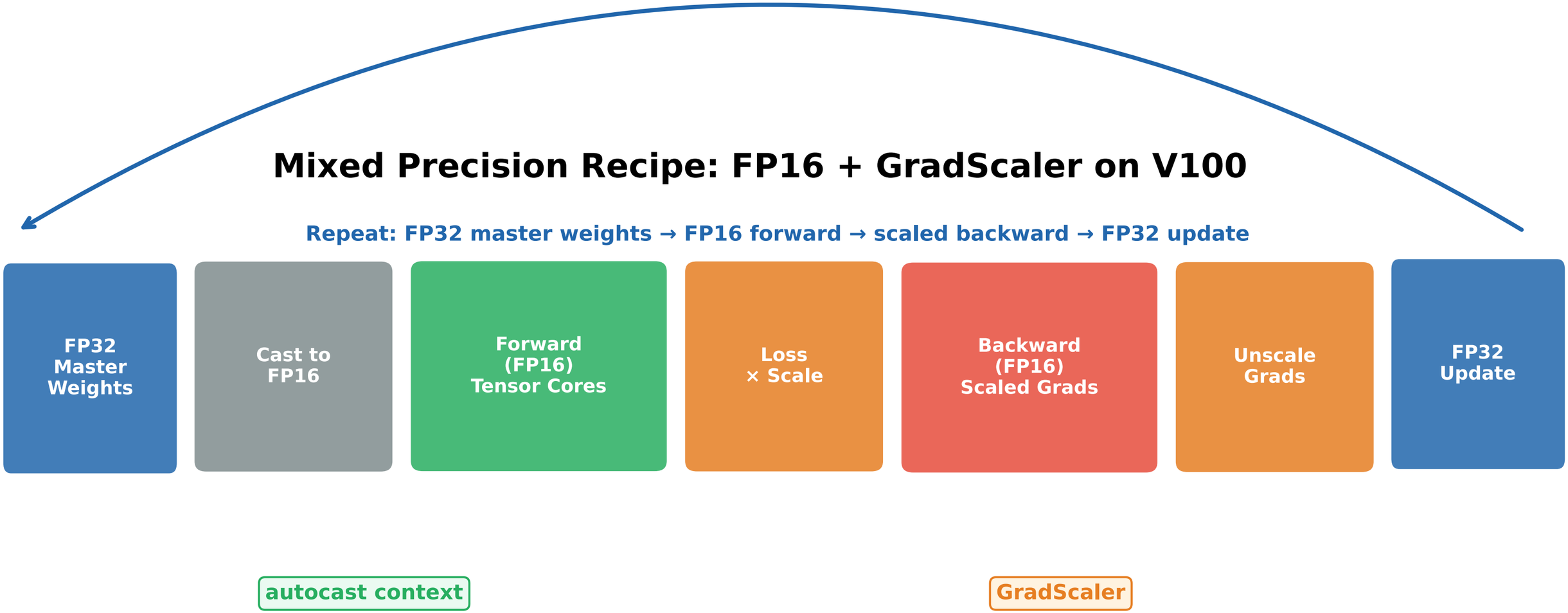
Fused Execution: 1 HBM Round-Trip (Efficient Attention / torch.compile)

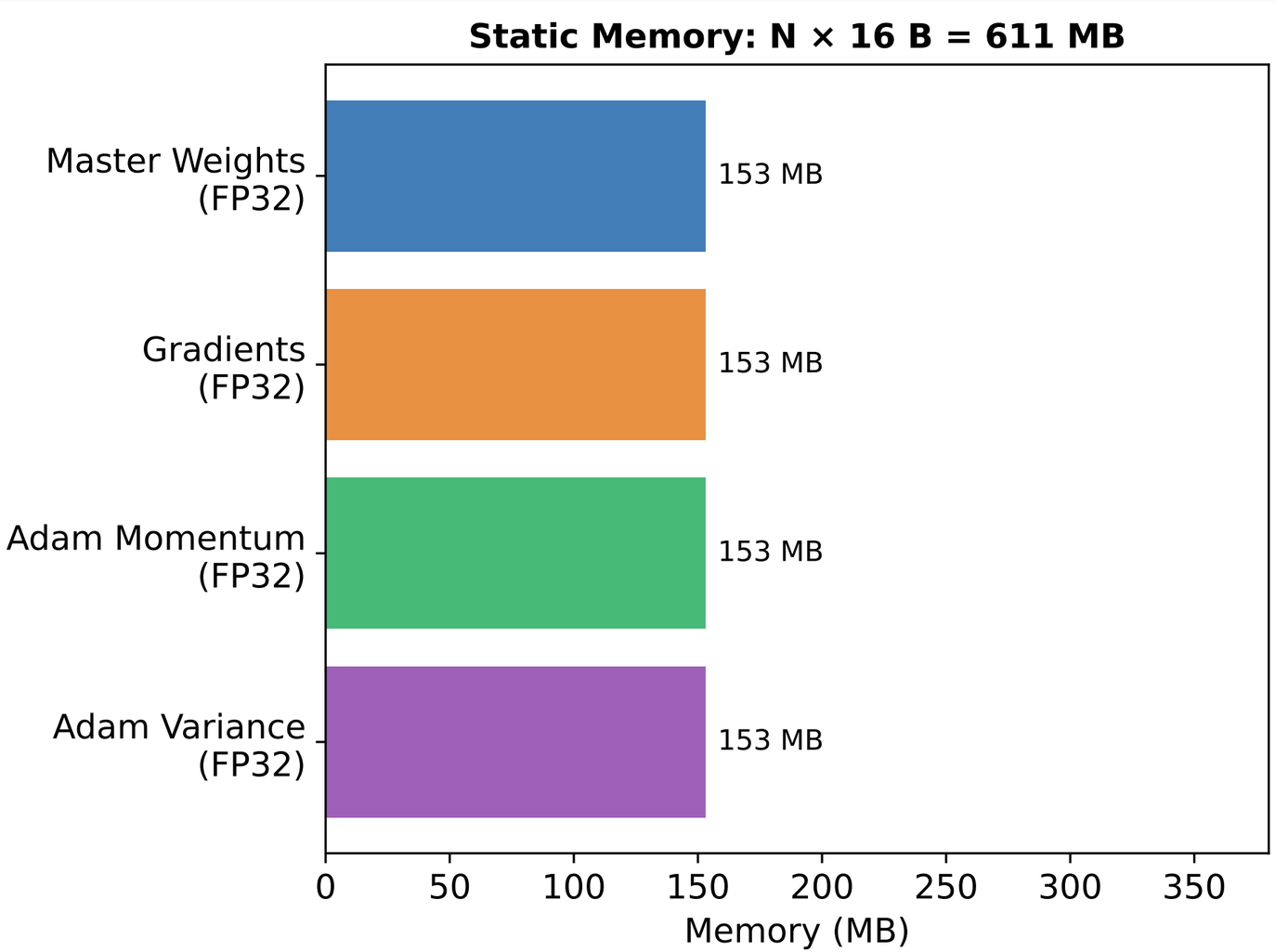


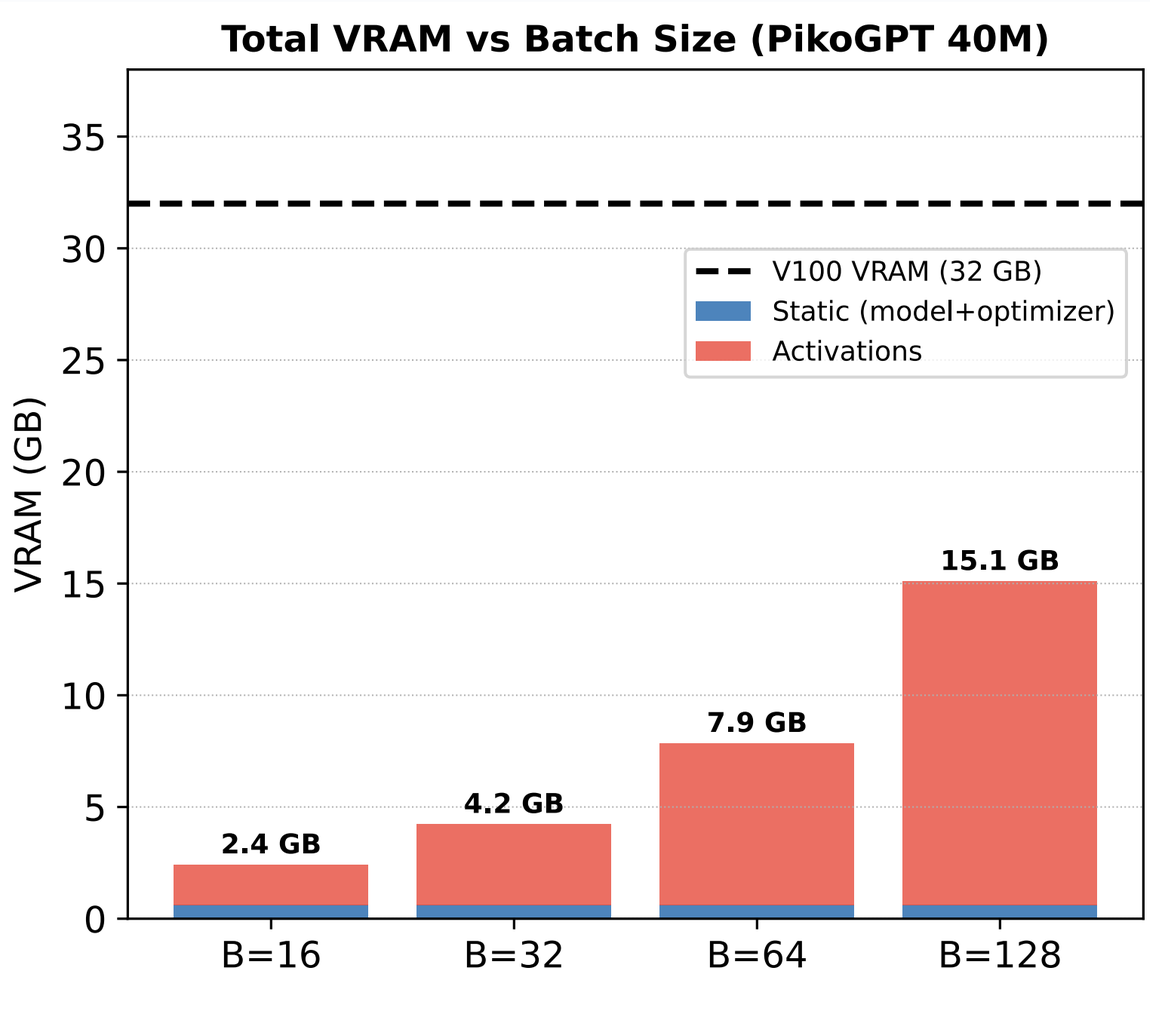
Floating Point Format Comparison



Mixed Precision Recipe: FP16 + GradScaler on V100




Static Memory: N × 16 B = 611 MB
Master Weights (FP32)
Gradients (FP32)
Adam Momentum (FP32)
Adam Variance (FP32)
153 MB
153 MB
153 MB
153 MB
0
50
100
150
200
250
300
350
Memory (MB)


Total VRAM vs Batch Size (PikoGPT 40M)
VRAM (GB)
0
5
10
15
20
25
30
35
V100 VRAM (32 GB)
Static (model+optimizer)
Activations
2.4 GB
4.2 GB
7.9 GB
15.1 GB
B=16
B=32
B=64
B=128

Take-Home Message: Making It Fast

Most ops are memory-bound

Only large matmuls are compute-bound. LayerNorm, softmax, GELU: the GPU waits for data.

Tiled Attention = 1.5–4× faster

Never writes the $T \times T$ matrix to HBM. Same math, less memory traffic. Use `scaled_dot_product_attention`.

FP16 + GradScaler on V100

8× faster matmul via Tensor Cores. GradScaler prevents FP16 underflow. Static memory: $N \times 16$ bytes.

Fuse operations, reduce HBM trips

`torch.compile` for auto-fusion (10-30% speedup). `scaled_dot_product_attention` for attention. Both are complementary.

Quick Check: Making It Fast

1. V100 balance point: ___ FLOPs/byte. Below = ___-bound.

2. Tiled attention reduces memory from $O(T^2)$ to $O(\_\_\_)$.

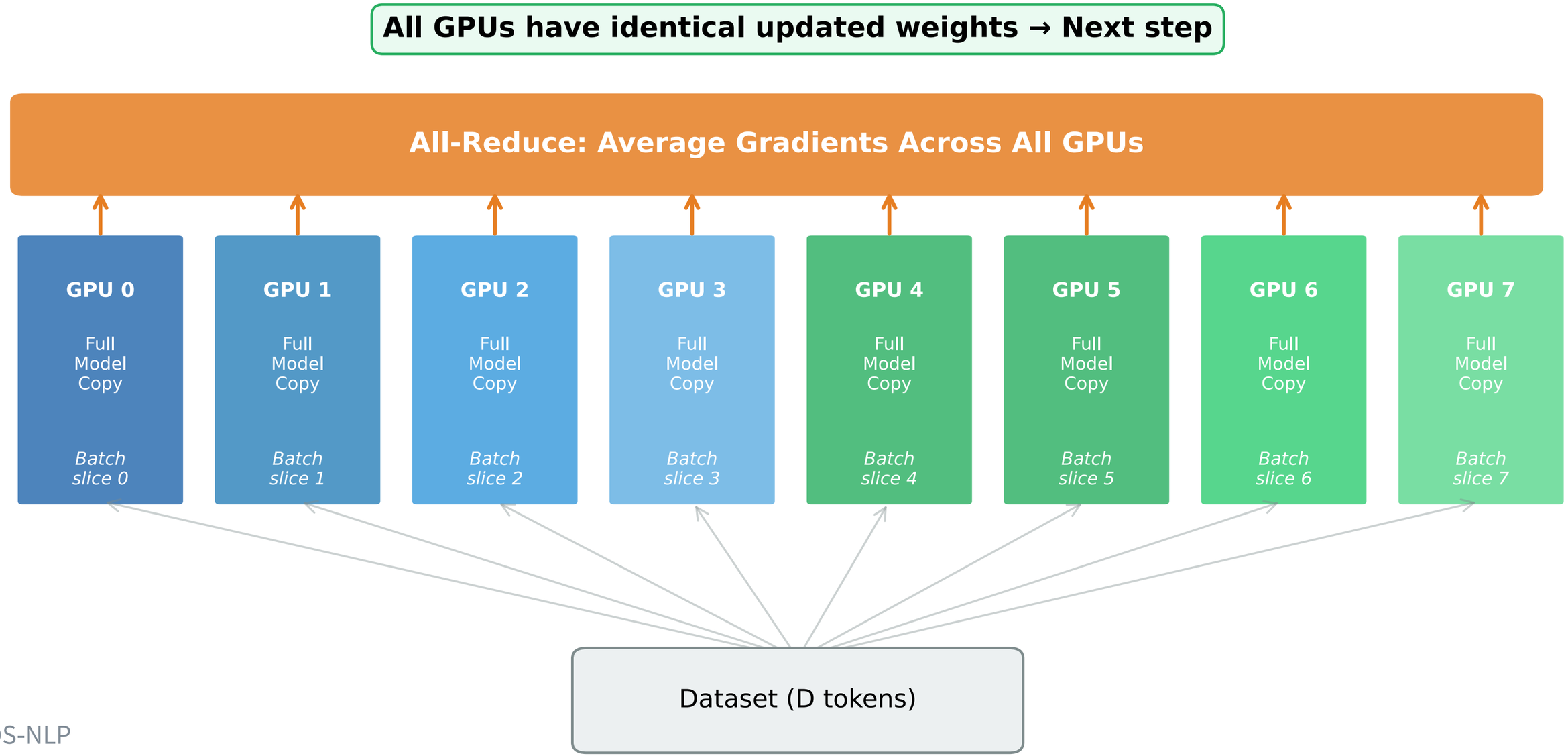
3. Why does V100 need GradScaler with FP16?

4. `torch.compile` gives ___% speedup by _____ operations.

Scaling Out

From 1 GPU to 8: DDP on the DGX-2

Distributed Data Parallel (DDP) on DGX-2: 8× V100



Data Parallelism: The Idea [5,6]

Same Model, Different Data

Each of the 8 GPUs gets:

1. A **full copy** of the model
2. A **different slice** of the batch
3. Computes forward + backward **independently**
4. **Synchronizes gradients** (all-reduce)
5. All GPUs make the **same weight update**

$$\text{effective batch} = B_{\text{per GPU}} \times N_{\text{GPUs}} \times K_{\text{accum}}$$

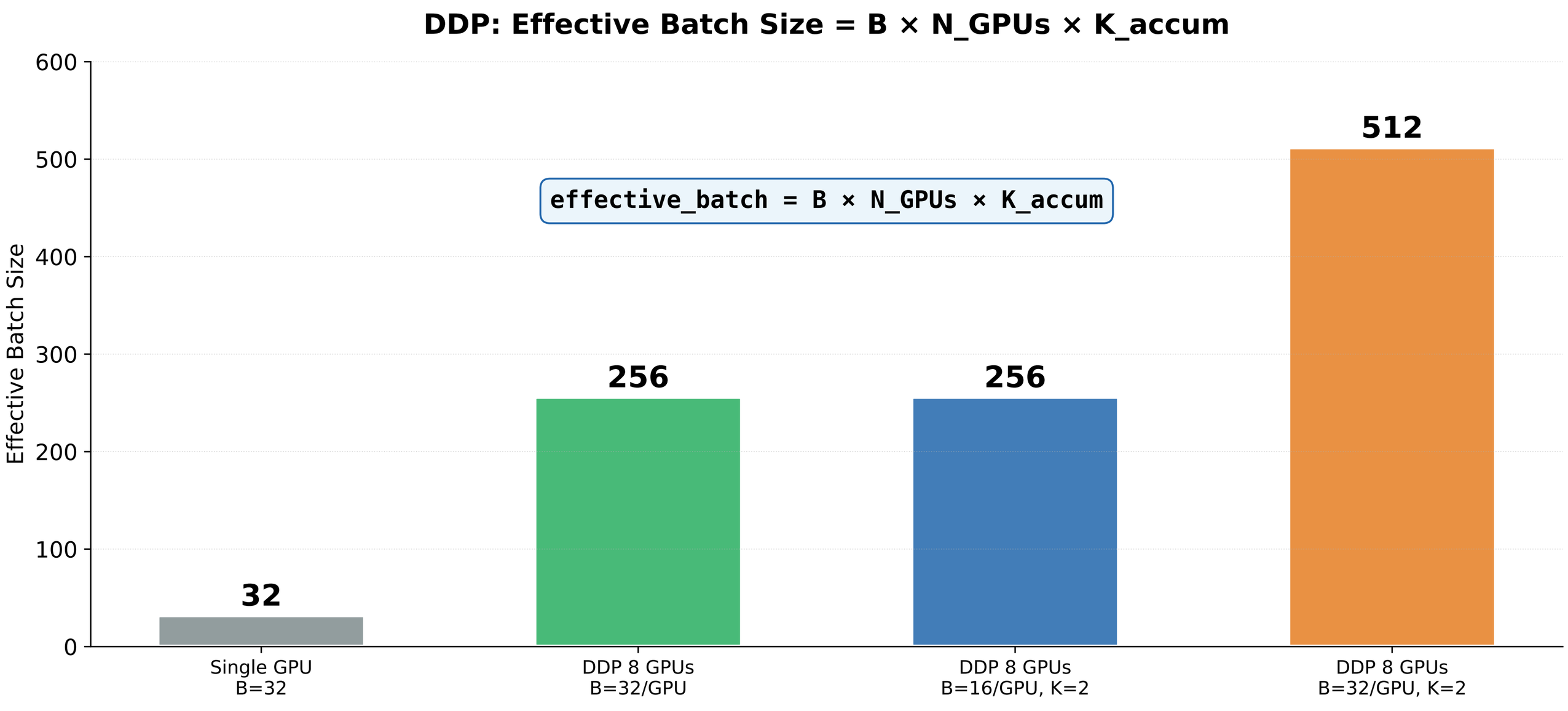
Example: $B = 16 \times 8GPUs \times 2 \rightarrow 256$ effective batch

Why It Works for PikoGPT

Question	Answer
Does the model fit on 1 GPU?	✅ 611 MB ≪ 32 GB
Do we want faster training?	✅ 8× compute
Is it simple to set up?	✅ ~5 lines of code

Data Parallelism is the **only** strategy you need for PikoGPT.

Model parallelism, pipeline parallelism, and tensor parallelism solve a *different* problem: models that **don't fit** on one GPU. Ours fits easily.


DDP: Effective Batch Size = B × N_GPUs × K_accum
effective_batch = B × N_GPUs × K_accum
Effective Batch Size
600
500
400
300
200
100
0
32
256
256
512
Single GPU
B=32
DDP 8 GPUs
B=32/GPU
DDP 8 GPUs
B=16/GPU, K=2
DDP 8 GPUs
B=32/GPU, K=2

The Batch Size Goldilocks Zone

The Accumulation Math

$$\text{Effective Batch} = B_{\text{GPU}} \times N_{\text{GPUs}} \times K_{\text{accum}}$$

How do we choose? There is no magic formula, only *empirical* history.

- **PikoGPT** is a smaller cousin of GPT-2.
- GPT-2 updated its weights after seeing roughly **500,000 tokens**.
- If our sequence length is 1024 tokens, $500,000 \div 1024 \approx \mathbf{500}$.
- Therefore, our target effective batch size is **512** (or 256 for faster iterations).

The Trade-Offs

Effective Batch	The Signal	The Risk
Too Small	Highly noisy	Erratic learning, hardware starved.
Sweet Spot	Clean & stable	Fast, efficient convergence.
Too Large	Perfectly smooth	Too few updates, gets stuck in local valleys.

The Golden Rule (Linear Scaling): If you double your effective batch size, you generally need to **double your learning rate**.

From 1 to 8 GPUs: The 3-Step Setup [5]

PyTorch handles the heavy lifting. You don't need to rewrite your math; you just need to organize the workers.

The Three Ingredients

1. The Network

`Init Process Group`

Opens the communication lines (NCCL) between the 8 GPUs so they can talk.

2. The Dealer

`DistributedSampler`

Shuffles the deck and deals a *unique* slice of the dataset to each GPU.

3. The Wrapper

`DDP(model)`

Wraps your model to automatically average everyone's math (gradients) in the background.

DDP in Action: The Untouched Loop & The Launch

Once the 3-step setup is complete, the actual execution and training logic is quite normal. PyTorch handles most of it in the background.

The Magic Loop

The core sequence (`forward` → `loss` → `backward` → `step`) remains **100% untouched**.

DDP invisibly hooks into the backward pass and averages the gradients across the NVLink bridge before taking a step.

The Classic Trap

You **must** tell the «Dealer» to shuffle every round:

```
sampler.set_epoch(epoch)
```

If you forget, every GPU sees the exact same data order forever. The model won't crash, but it will overfit wildly.

The Launch Code

Instead of running standard Python, use the PyTorch distributed launcher:

```
torchrun --nproc_per_node=8 train.py
```

(This automatically spawns 8 clones of your script, assigning each a `LOCAL_RANK` from 0 to 7).

When the Model is Too Big: Free Lunch vs. Brain Surgery [7]

When a model physically cannot fit you must stop replicating and start **sharding** (chopping it into pieces).

🍔 The «Free Lunch» (Data Parallel)

PyTorch handles this under the hood

Strategy	The Intuition
DDP	**Clone everything.** Split the data. *(Our approach)*
ZeRO / FSDP[1]	**Shatter the memory.** GPUs hold fractions of the model and pass pieces around on the fly.

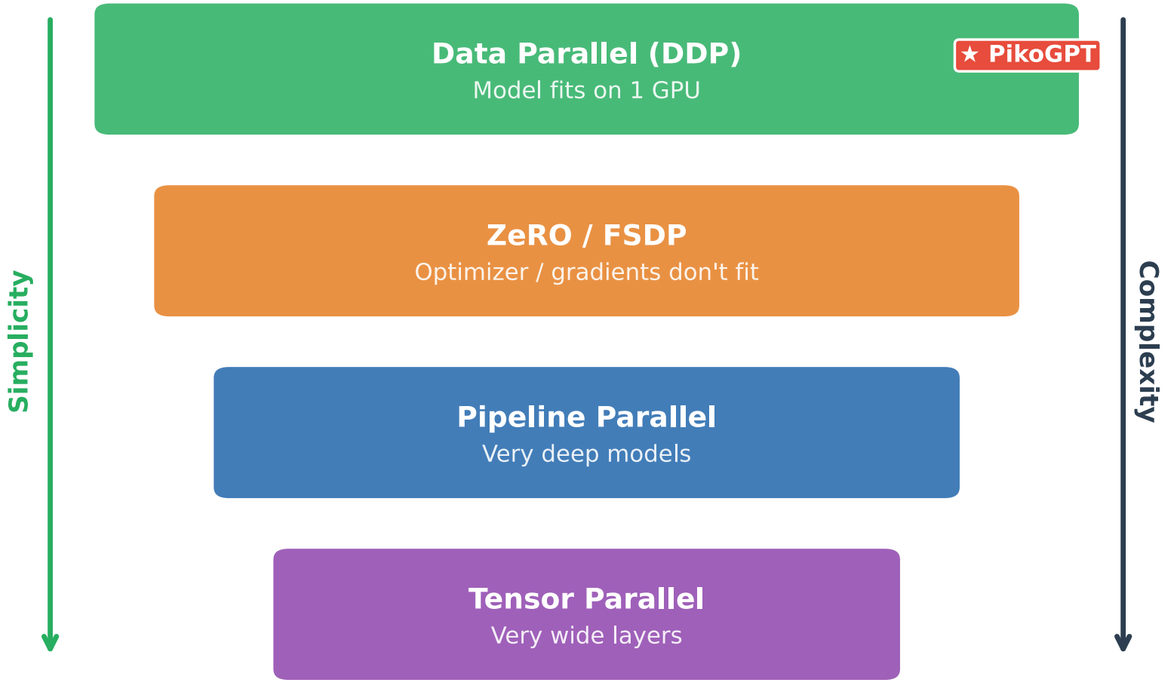
🧠 «Brain Surgery» (Model Parallel)

Manually rewrite the architecture and physically slice the network.

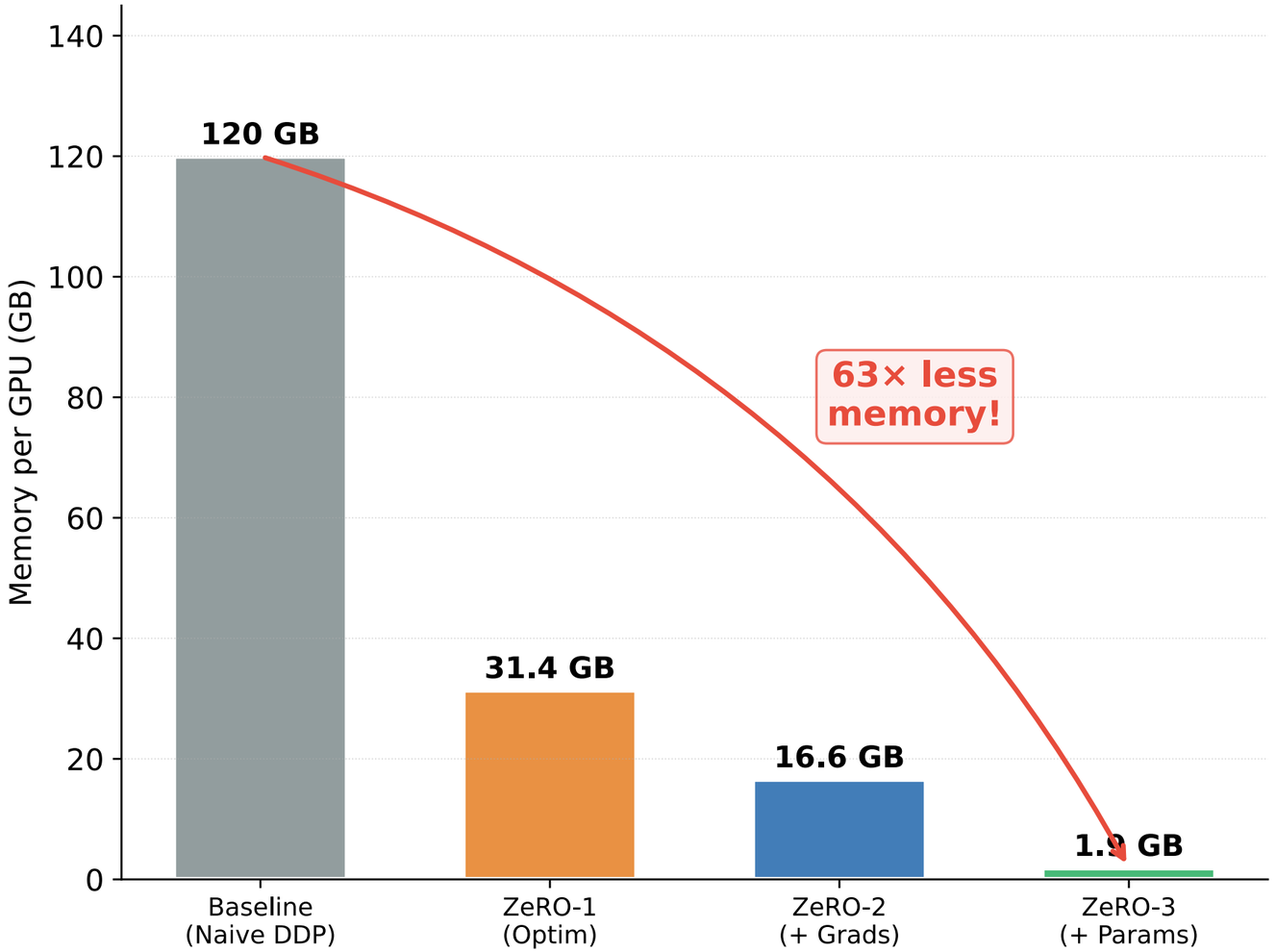
Strategy	The Intuition
Pipeline Parallel	**The Assembly Line.** GPU 1 physically holds layers 1-10; GPU 2 holds 11-20.
Tensor Parallel	**Micro-surgery.** Splitting a single massive matrix multiplication across multiple GPUs.

[1] FSDP = Fully Sharded Data Parallel (PyTorch's native ZeRO-3 implementation)

Parallelism Strategies: When to Use What



ZeRO: Memory Savings (7.5B Model, 64 GPUs)



Profiling 101: The "Stethoscope" Check [1]

Check the dashboard. `nvidia-smi`

📉 The Starving GPU

Symptom: `GPU-Util < 50%`

Diagnosis: Your supercomputer is bored. It processes math faster than your CPU can load the text files.

The Fix: Increase `num_workers` and enable `pin_memory` in your DataLoader.

💥 The Memory Crash

Symptom: `CUDA Out of Memory`

Diagnosis: You tried to cram too many intermediate math steps (activations) into VRAM at once.

The Fix: Shrink your physical Batch Size and use **Gradient Accumulation**.

🐢 The Sluggish Math

Symptom: High Util, but slow steps.

Diagnosis: The GPU is busy, but doing math the slow, brute-force way.

The Fix: Turn on `torch.compile` and ensure FlashAttention is active.

The Deep MRI: PyTorch Profiler [1]

What it Measures

The profiler wraps a few training steps and outputs a timeline trace.

- **Time per Operation:** Is the bottleneck the `matmul`, the `softmax`, or the `LayerNorm`?
- **CUDA vs. CPU Time:** Is the GPU constantly stalling because it has to wait for the CPU to send the next instruction?
- **Memory Spikes:** Exactly which layer causes your VRAM to suddenly spike and crash?

The Golden Rule of Optimization

Strategy	Focus
1. Find the tallest tentpole	Sort operations by total CUDA time.
2. Fix the worst offender	Optimize *only* that specific block.
3. Repeat	Profile again. The bottleneck will have moved.

(Detailed profiler implementation awaits in EX06).

The Hidden Bottleneck: Data Loading

The Problem

Your GPU computes a training step in **milliseconds**. But loading the next batch from disk can take **seconds**. The GPU sits idle, waiting.

Symptom: `nvidia-smi` shows GPU-Util at 20-40% despite a large model and batch.

If your GPU utilization is below 80%, data loading is almost certainly the bottleneck.

The Fix: Parallel Data Loading

Setting	What It Does	Recommendation
`num_workers`	CPU threads loading data in parallel	Start with 4, try 8
`pin_memory`	Faster CPU→GPU transfer via page-locked RAM	Always `True`
`prefetch_factor`	Pre-load next batches while GPU trains	2 (default)

The idea: While the GPU processes batch n, the CPU is already loading batch $n + 1$ from disk. With enough workers, the GPU never waits.

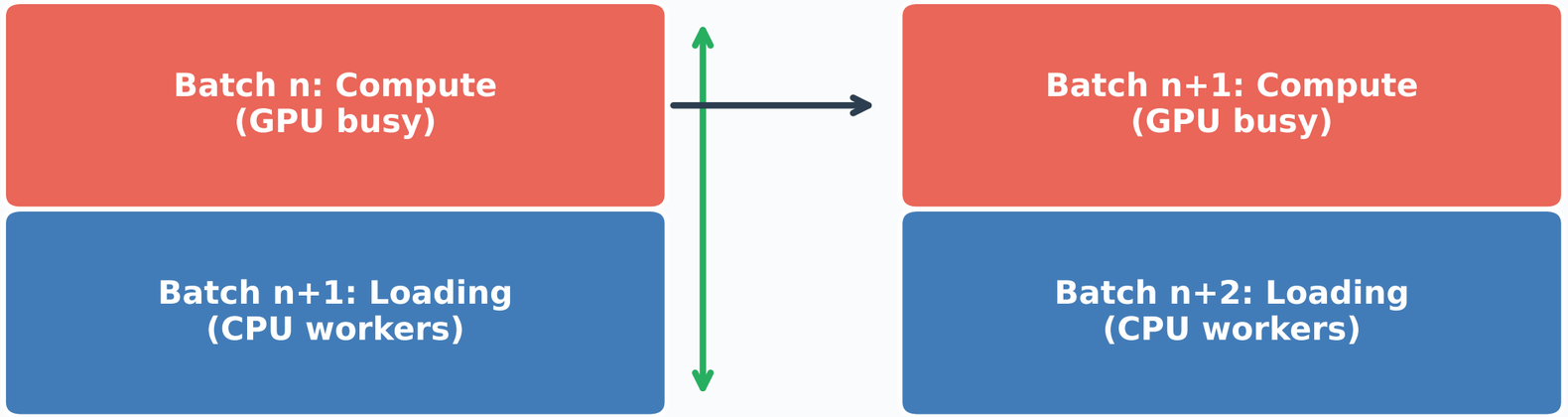
On DGX-2: Each V100 process needs its own workers. With 8 GPUs and `num_workers=4`: 32 CPU threads loading data in

Data Loading Pipeline: Keep the GPU Fed

Without Prefetch:

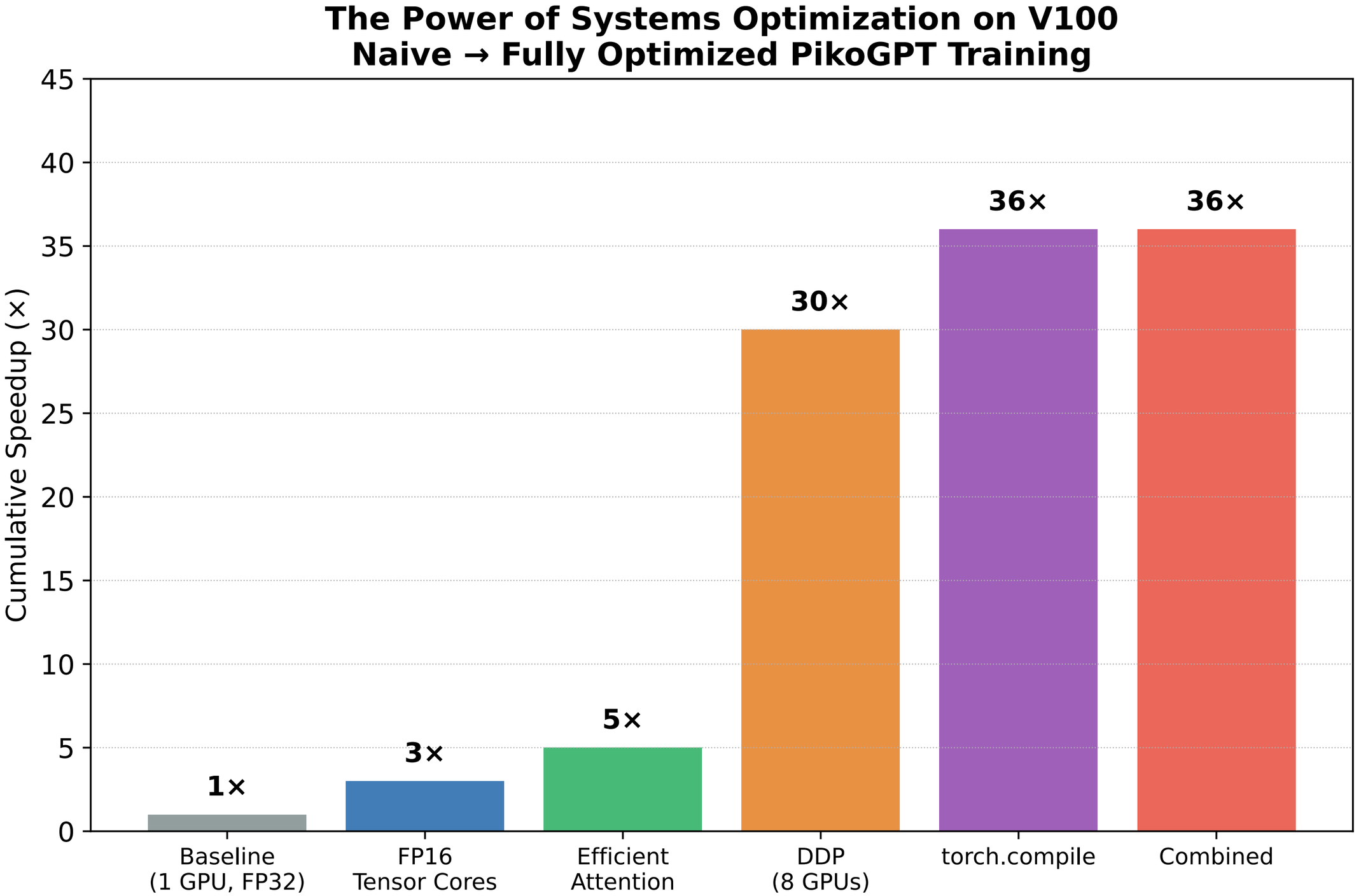
With Prefetch:

Overlapped! GPU never waits



Key Settings:

```
num_workers = 4+
pin_memory = True
prefetch_factor = 2
```


The Power of Systems Optimization on V100
Naive → Fully Optimized PikoGPT Training
Cumulative Speedup (×)
0
5
10
15
20
25
30
35
40
45
1×
3×
5×
30×
36×
36×
Baseline
(1 GPU, FP32)
FP16
Tensor Cores
Efficient
Attention
DDP
(8 GPUs)
torch.compile
Combined

Putting It All Together: Demo Model (120M) on the DGX-2

Parameter	Value
Model size (N)	120M
Data ratio (f)	20
Tokens ($D = fN$)	2.4B
Compute ($C = 6ND$)	1.73×10^{18} (Exa)

Resource	Specification
GPU	8× V100 (32 GB each)
Throughput / GPU	~40 TFLOPs (FP16)
Static memory	~1.9 GB ≪ 32 GB
Activations (B=32)	Fits comfortably

The Calculation

Single GPU, FP32 (naive):

$$T = \frac{1.73 \times 10^{18}}{15.7 \times 10^{12}} \approx 110{,}000\text{s} \approx \mathbf{30.6\ hours}$$

Single GPU, FP16 (Tensor Cores):

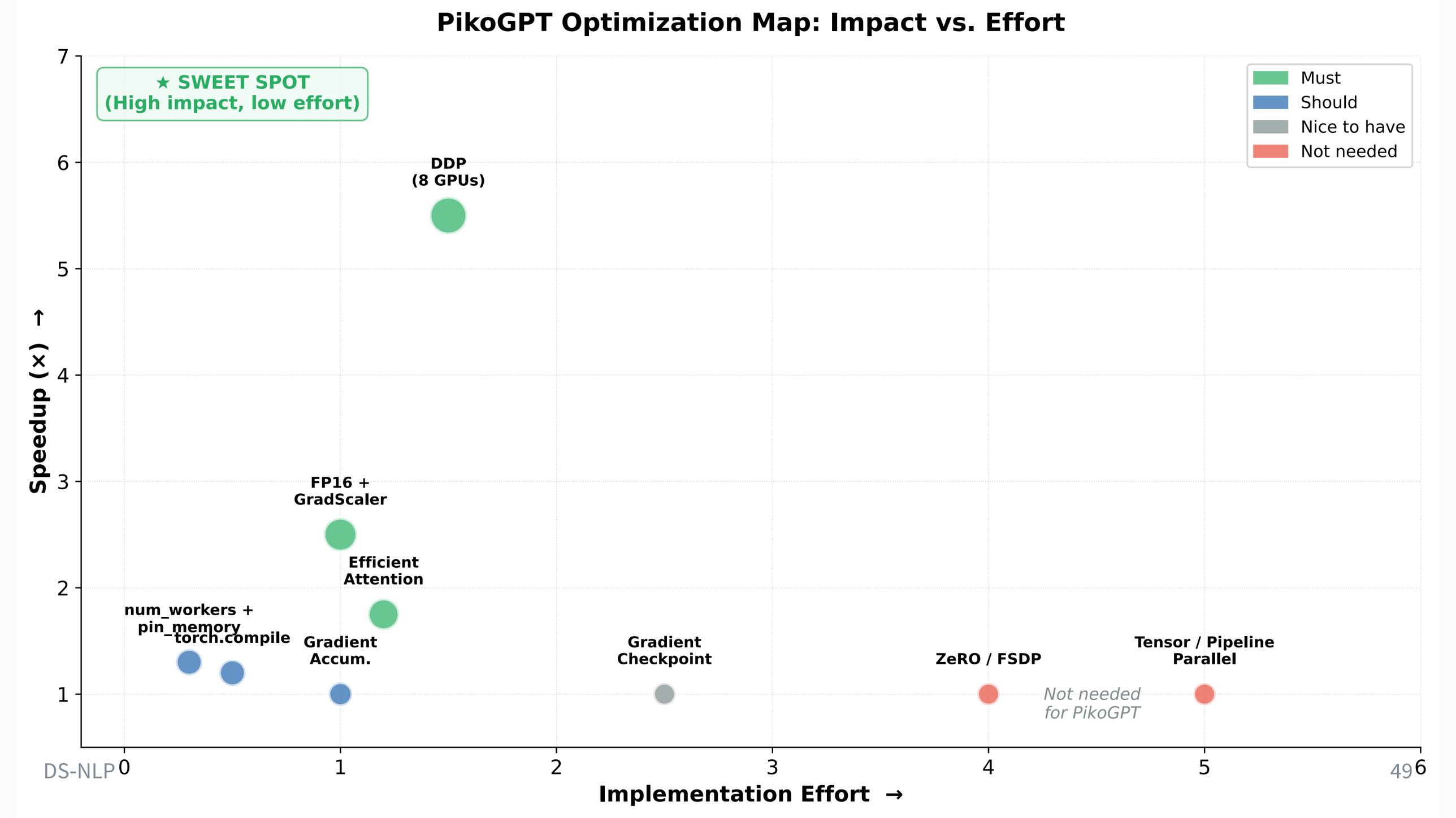
$$T = \frac{1.73 \times 10^{18}}{40 \times 10^{12}} \approx 43{,}200\text{s} \approx \mathbf{12\ hours}$$

8× GPU, FP16 + DDP (~6× scaling):

$$T \approx \frac{12\ \text{hours}}{6} \approx \mathbf{2\ hours}$$

From 30.6 hours to 2 hours: same model, same data, just better systems.

Add torch.compile, efficient attention, and real overhead: expect ~2.5 - 3 hours for the full run.


PikoGPT Optimization Map: Impact vs. Effort
★ SWEET SPOT
(High impact, low effort)
Must
Should
Nice to have
Not needed
DDP
(8 GPUs)
FP16 +
GradScaler
Efficient
Attention
num_workers +
pin_memory
torch.compile
Gradient
Accum.
Gradient
Checkpoint
ZeRO / FSDP
Tensor / Pipeline
Parallel
Not needed
for PikoGPT
Speedup (×) →
Implementation Effort →
0
1
2
3
4
5
6
7

Take-Home Message: Scaling Out

DDP is all you need (for PikoGPT)

Model fits on 1 GPU → replicate on 8, split the batch. 5 lines of code. ~6× speedup.

Profile before optimizing

`nvidia-smi` for GPU-Util, `torch.profiler` for details. Low utilization usually = data loading bottleneck.

The Big Four: FP16 + SDPA + DDP + compile

Combined ~30× over naive baseline on V100. Non-negotiable for serious training.

ZeRO, FSDP, TP, PP: know they exist

These solve a different problem: models that don't fit on one GPU. Not your problem with 40M params.

Quick Check: Scaling Out

1. DDP with B=16, 8 GPUs, K=2: effective batch = ___.

3. GPU-Util shows 30%. Most likely bottleneck?

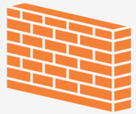
Summary & Outlook

Summary

1. GPU Architecture

GPUs are **throughput machines**: thousands of cores, memory hierarchy from registers to HBM.

Tensor Cores make FP16 **8× faster** than FP32.



2. The Memory Wall

Most Transformer ops are **memory-bound**.

Tiled attention + kernel fusion reduce HBM traffic. `scaled_dot_product_attention` auto-selects the best V100 kernel.

3. Mixed Precision (V100)

FP16 + GradScaler is mandatory on V100.

Static memory: $N \times 16$ bytes. Activations dominate.

4. DDP + Profiling

Data Parallelism: 5 lines, ~6× speedup.

Profile first, then optimize. The Big Four: FP16 + SDPA + DDP + compile ≈ 30× on V100.

Next: Semester Break → Pre-training runs.
Week 7: **Midterm Presentation** + Post-Training I (SFT).

Kahoot Quiz: Active Recall

Kahoot Quiz VL06

play.kahoot.it

Optimization Checklist: Low-Hanging Fruit

Priority	Optimization	Speedup	Effort	PikoGPT?
Must	Mixed precision (FP16 + GradScaler)	2-3×	Low	✅
Must	Efficient Attention (`scaled_dot_product_attention`)	1.5–2× (V100)	Low	✅
Must	DDP on 8 GPUs	5-6×	Low	✅
Should	`torch.compile`	10-30%	1 line	✅
Should	Increase `num_workers` + `pin_memory`	10-50%	1 line	✅
Should	Gradient accumulation (if B too large)	—	Low	If needed
Nice	Gradient checkpointing	Saves ~60% act. mem	Medium	Only if OOM
Not needed	ZeRO / FSDP	Memory savings	High	❌ (model tiny)
Not needed	Tensor / Pipeline Parallel	Huge models only	High	❌

Combined: FP16 (3×) × Efficient Attn (~1.7×) × DDP (6×) × compile (1.2×) ≈ **~30× faster** than naive single-GPU FP32 baseline on V100.

Pre-Training Readiness Checklist

Code Ready?

Check	Status
Tokenizer loads & encodes correctly	☐
Model forward pass runs (B=1, T=1024)	☐
Loss decreases on toy data (1M tokens)	☐
Mixed precision (FP16 + GradScaler) works	☐
`scaled_dot_product_attention` enabled	☐
`torch.compile` applied	☐
DDP works on 2+ GPUs	☐
Checkpointing saves & loads	☐

Scaling Plan Ready?

Check	Status
Model size N chosen (≤40M)	☐
Data ratio f chosen ($f \in [5, 20]$)	☐
$D = f \times N$ computed	☐
$C = 6ND$ computed	☐
Wall-clock time estimated	☐
Memory budget verified (fits V100 32GB)	☐
LR sweep done (3-5 rates at small scale)	☐
Architecture finalized (L, d, heads)	☐

If any box is unchecked → fix it this week. The break is for **training**, not debugging.

Common Pitfalls (and How to Avoid Them)

Pitfall	Symptom	Prevention
No checkpointing	24h run crashes at hour 20, all lost	Save every 500-1000 steps + test resume
Wrong LR	Loss explodes or barely moves	LR sweep at 1/10 scale before full run
No GradScaler on V100	Loss is NaN, gradients underflow	Always use `torch.GradScaler()` with FP16
Data loading bottleneck	GPU-Util <50%, step time varies	`num_workers=4+`, `pin_memory=True`
Forgetting `set_epoch()`	Subtle overfitting, same data order	Always `sampler.set_epoch(epoch)` in DDP
Not using SDPA	OOM at T=1024, slow training	Use `F.scaled_dot_product_attention()`
Batch too large	OOM	Start small (B=16), increase gradually
No validation	Don't notice overfitting/data issues	Validate every 500 steps on held-out data

Test your full pipeline (including checkpoint save/load) at small scale before the break.

Key Formulas & Numbers

Formula / Number	Meaning	PikoGPT Value
V100 FP16: 125 TFLOPs	Peak Tensor Core throughput	MFU ~30-35% → ~40 TFLOPs effective
V100 HBM: 900 GB/s, 32 GB	Memory bandwidth and capacity	Our GPU
Balance point: ~139 FLOPs/byte	Above → compute-bound, below → memory-bound	V100 specific
$\text{Static Memory} = N \times 16\text{B}$	Weights (4B) + Grads (4B) + Adam (8B)	40M × 16 = 611 MB
$\text{Activations} \approx 12 \times B \times T \times d \times L \times 2\text{B}$	Per-step activation memory (FP16)	B=64: ~14.4 GB
$\text{effective\_batch} = B \times N_{\text{GPU}} \times K$	DDP effective batch size	32 × 8 × 1 = 256
$\text{All-reduce cost} \approx 2 \times N \times 4\text{B}$	DDP communication per step	~320 MB (~1 ms on NVLink)

Glossary (1/3)

Term	Definition
All-Reduce	Collective operation that averages tensors across all GPUs. Used in DDP to synchronize gradients. Communication cost: ~2× tensor size.
Arithmetic Intensity	Ratio of FLOPs to bytes loaded from memory. Determines whether an operation is compute-bound (high) or memory-bound (low).
BF16 (BFloat16)	16-bit format with 8-bit exponent (same range as FP32) and 7-bit mantissa. Supported on A100/H100 but NOT on V100. Does not need GradScaler.
Compute-Bound	An operation where the GPU's compute units are the bottleneck. Large matrix multiplications are typically compute-bound.
CUDA Cores	General-purpose GPU processing units. V100 has 5,120 FP32 CUDA cores. Slower than Tensor Cores for matrix operations.
DDP (Distributed Data Parallel)	PyTorch data parallelism: replicate model, split batch, all-reduce gradients. The standard approach when model fits on one GPU. [5]
FP16 (Float16)	16-bit format with 5-bit exponent (range ±65,504) and 10-bit mantissa. Fast on V100 Tensor Cores. Requires GradScaler.

Glossary (2/3)

Term	Definition
FSDP (Fully Sharded Data Parallel)	PyTorch implementation of ZeRO-3. Shards parameters, gradients, and optimizer states across GPUs. For models that don't fit on one GPU.
Gradient Accumulation	Simulating larger batch sizes by summing gradients over K micro-batches before an optimizer step. $\text{effective\_batch} = B \times N_{\text{GPU}} \times K$.
Gradient Checkpointing	Trading compute for memory: recompute activations during backward instead of storing them. ~33% slower, ~60% less activation memory.
GradScaler	PyTorch utility for FP16 on V100. Scales loss up before backward (prevents underflow), unscales gradients before optimizer step. Dynamic scale factor. [8]
HBM (High Bandwidth Memory)	Main GPU memory (VRAM). V100: 32 GB at 900 GB/s. Slowest on-chip memory (~290 cycles). Where weights, gradients, and activations live.
Kernel Fusion	Combining multiple GPU operations into a single kernel launch. Data stays in SRAM instead of round-tripping through HBM. `torch.compile` does this automatically.
Memory-Bound	An operation where HBM bandwidth is the bottleneck. Most non-matmul Transformer operations are memory-bound.

Glossary (3/3)

Term	Definition
MFU (Model FLOPs Utilization)	Fraction of theoretical peak GPU FLOPs actually achieved. V100 FP16: 125 TFLOPs peak, ~30-35% MFU = ~40 TFLOPs effective.
Mixed Precision	FP16 forward/backward (Tensor Cores) + FP32 master weights and optimizer. On V100: requires GradScaler. Static memory: $N \times 16$ bytes.
NCCL	NVIDIA Collective Communications Library. Fastest backend for GPU-to-GPU communication. Uses NVLink. Always use `backend='nccl'` for DDP.
NVLink	High-speed GPU-to-GPU interconnect. DGX-2 V100: ~300 GB/s. Much faster than PCIe (~32 GB/s).
Pipeline Parallelism	Distributes layers across GPUs. Requires micro-batching. For very deep models that don't fit on one GPU.
SM (Streaming Multiprocessor)	Fundamental GPU compute unit. V100: 80 SMs, each with 64 FP32 cores + 8 Tensor Cores + 96 KB shared memory.
SRAM (Shared Memory)	Fast on-die memory inside each SM. V100: 96 KB/SM. ~8× faster than HBM. Tiled attention works by keeping computation in SRAM.
Tensor Cores	Specialized matmul circuits (V100+). V100 FP16: 125 TFLOPs vs 15.7 on CUDA cores. Require FP16/BF16 input.

Bibliography

[1] Stanford CS336 (2025). Language Modeling from Scratch — Lecture 5: GPUs. https://stanford-cs336.github.io/spring2025/

[2] NVIDIA (2017). Tesla V100 GPU Architecture Whitepaper. https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf

[3] He, H. (2022). Making Deep Learning Go Brrrr From First Principles. https://horace.io/brrr_intro.html

[4] Dao, T. et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS / arXiv:2205.14135.

[5] Li, S. et al. (2020). PyTorch Distributed: Experiences on Accelerating Data Parallel Training. VLDB / arXiv:2006.15704.

[6] Stanford CS336 (2025). Language Modeling from Scratch — Lecture 7: Parallelism Basics. https://stanford-cs336.github.io/spring2025/

Bibliography (cont.)

[7] Rajbhandari, S. et al. (2020). ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. SC / arXiv:1910.02054.

[8] Micikevicius, P. et al. (2018). Mixed Precision Training. ICLR / arXiv:1710.03740.